



Operating Systems

ADAPTIVE TASK DECOMPOSITION AND SCHEDULING

WITH AI-ASSISTED THREAD ALLOCATION

ROLL NO.	PRN	Name
8	12311740	Dhruv Kulkarni
18	12311548	Mandar Chinnapurkar
20	12310209	Mayank Bhagwat

Div: B-B1

Group No. 02

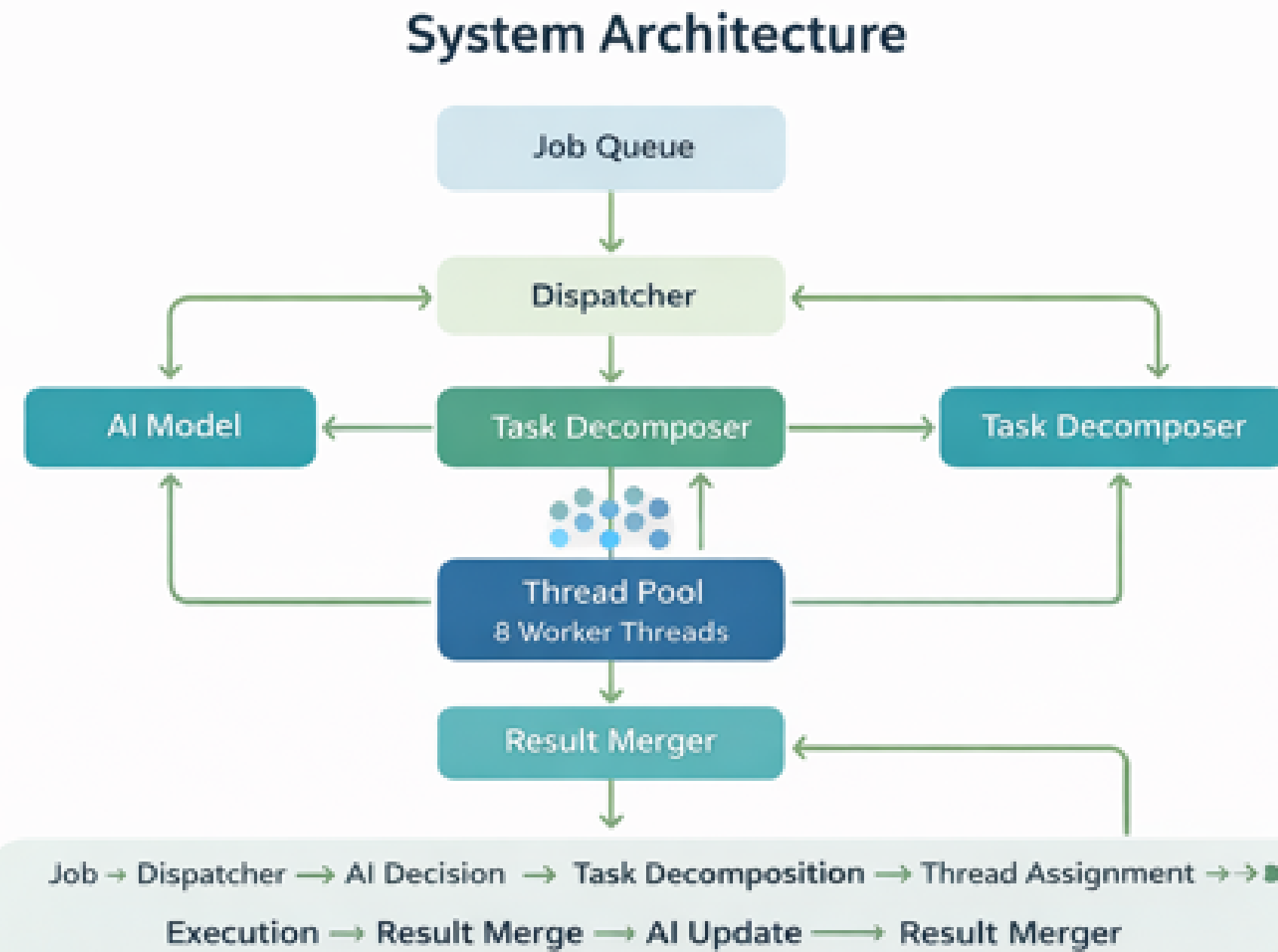
Guide:

Prof. Abha Marathe

Problem Statement & Proposed Solution

- Traditional schedulers use fixed threads for all tasks
 - Over-threading small tasks → overhead
 - Under-utilization for large tasks → slower execution
 - No learning from past executions
-
- Adaptive multi-threaded scheduler
 - Dynamically decomposes tasks into subtasks
 - Uses AI to decide optimal thread allocation
 - Continuously improves performance over time

System Architecture



AI Model & Novelty

AI Model

- Learns optimal thread count based on:
 - Process type
 - Workload size
- Uses historical execution data

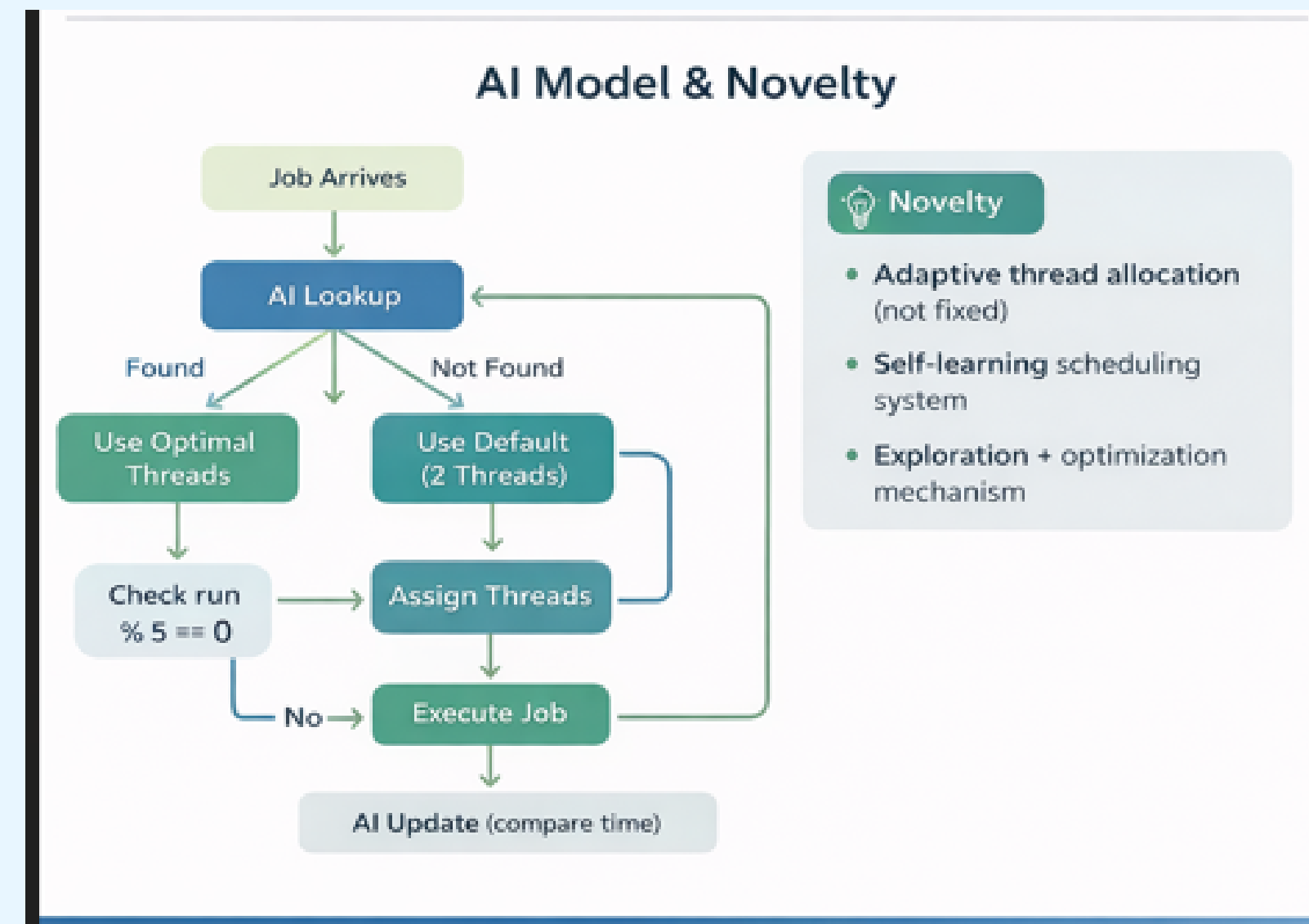
Key Functions

- `ai_decide()` → selects thread count
- `ai_update()` → updates best performance
- `ai_lookup()` → retrieves past data

Learning Strategy

- Default: 2 threads
- Every 5th run → explores higher threads
- Updates if execution is faster

- Adaptive thread allocation (not fixed)
- Self-learning scheduling system
- Exploration + optimization mechanism
- Works with real OS system calls



Task Decomposition & Execution

Task Decomposition

- Each job is split into N subtasks
- Each subtask assigned a data range

Examples

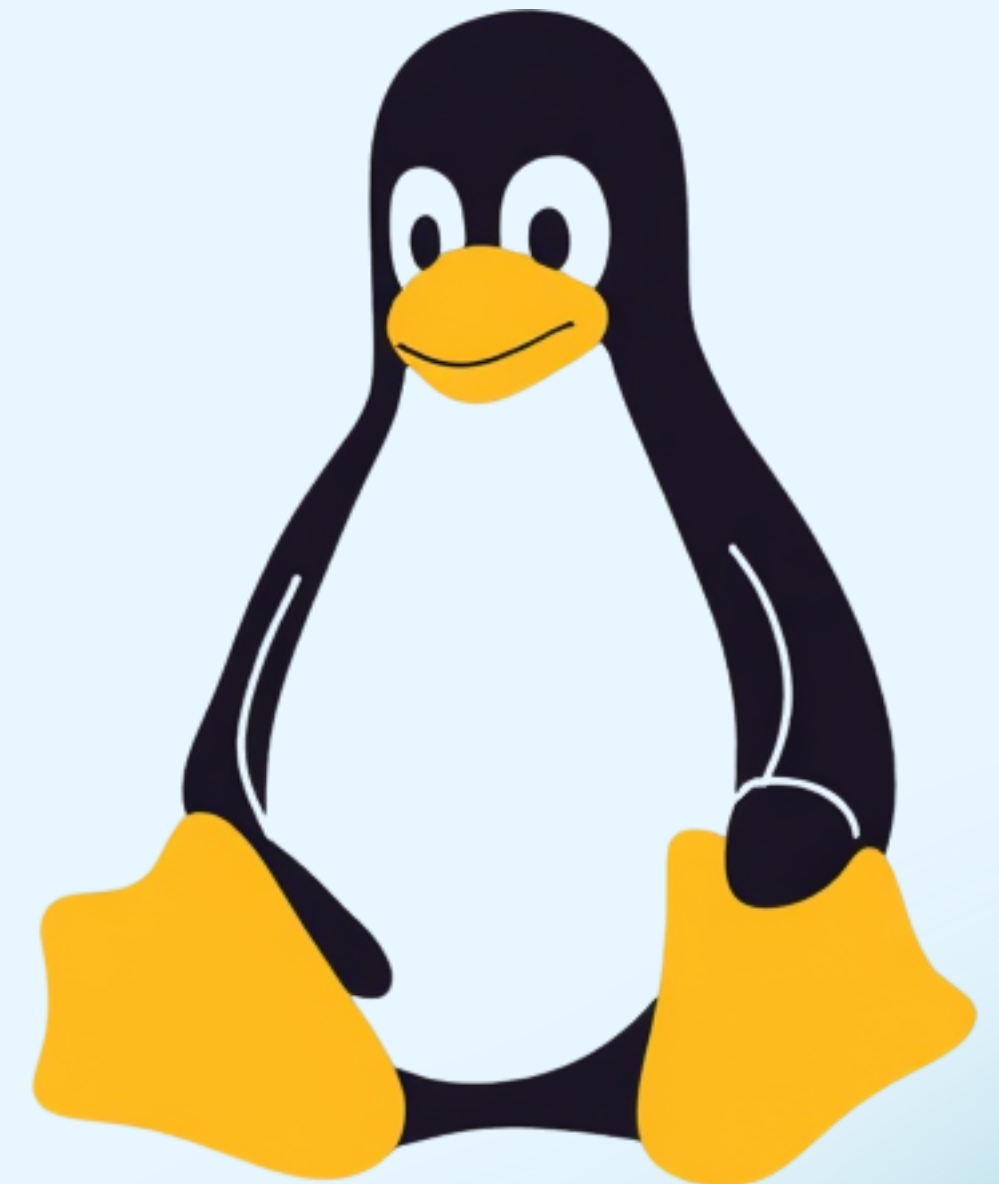
- File Copy → split by bytes
- File Search → split by folders
- Log Analysis → split by lines

Execution

- Worker threads execute subtasks
- Uses POSIX system calls:
 - `open()`, `read()`, `write()`, `close()`

Result

- Parallel execution improves performance



Results & Performance

Without AI

- Fixed thread allocation
- Inefficient resource usage

With AI

- Learns optimal threads
- Reduces execution time
- Improves consistency
- Performance Improvement

Example:

- Round 1 → 520ms
- Round 5 → 385ms

~24% improvement

Key Benefits

- Better CPU utilization
- Faster execution
- Adaptive optimization

Conclusion

- Adaptive scheduling improves performance
- AI enables intelligent resource allocation
- System becomes efficient over time
- Applications
- Operating systems
- Cloud scheduling
- Distributed systems

SCHEDULING ALGORITHMS

1. FCFS (First-Come, First-Served)



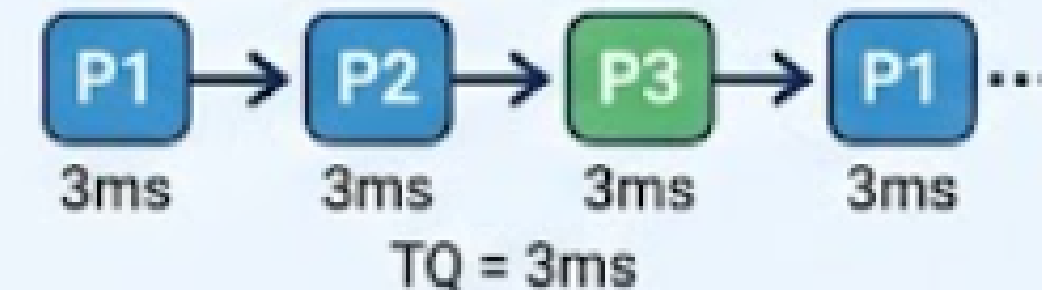
2. SJF (Shortest Job First)



3. PRIORITY SCHEDULING



4. ROUND ROBIN (RR)





THANK YOU